AI-SUPPORTED OBJECT TRACKING SYSTEM

A DEEP LEARNING AND IMAGE PROCESSING APPROACH ON JETSON NANO

Refiye Deniz BATTAL

Zonguldak Bülent Ecevit University

Department of Electrical and Electronics Engineering

231106203016

rdeniz.battal@mf.karaelmas.beun.edu.tr

Suat AYKURT

Zonguldak Bülent Ecevit University

Department of Electrical and Electronics Engineering

200106106076

s.aykurt@mf.karaelmas.beun.edu.tr

Abstract: In this project, an AI-supported object tracking system based on the NVIDIA Jetson Nano was designed and implemented. The system utilizes the YOLOv5s object detection algorithm to identify objects in the images captured via a USB camera and tracks the identified target using a two-axis (pan-tilt) servo motor mechanism controlled by a PCA9685 driver board. The control mechanism of the servo motors, the features of the driver board, and its performance were examined in detail. Test results indicate that the system operates in real time and successfully tracks target objects.

Keywords: Servo Motor Control, SG90, PCA9685, PWM Driver, Jetson Nano, Object Tracking, YOLOv5.

I. INTRODUCTION

With the advancement of artificial intelligence and image processing technologies, it has become possible for autonomous systems to perform real-time target detection and tracking. These systems have found wide applications in security, robotics, and industrial fields [1]. One of the main components of such systems is the servo motor mechanisms that enable the physical tracking of the target object.

In the developed project, a system was built that runs on NVIDIA Jetson Nano, detects objects using the YOLOv5s algorithm, and tracks the detected objects with a two-axis servo motor mechanism. One of the system's most critical components, the servo motor control mechanism, was implemented using the PCA9685 PWM driver board.

II. SERVO MOTOR SYSTEMS

A. Operating Principle of Servo Motors

Servo motors are feedback-based electromechanical systems that provide precise position control. Their basic structure consists of a DC motor, a gear mechanism, a position sensor (typically a potentiometer), and a control circuit. Servo motors are controlled using PWM (Pulse Width Modulation) signals, where the pulse width determines the motor's position [2].

For a standard analog servo motor:

- 1 ms pulse width: -90° position
- 1.5 ms pulse width: 0° position (center)
- 2 ms pulse width: +90° position

Servo motors are typically controlled with PWM signals at a frequency of 50 Hz (20 ms period). Lee et al. [3] demonstrated that the positional accuracy of servo motors is directly dependent on the PWM frequency and voltage stability.

B. SG90 Servo Motors

In the developed system, SG90 micro servo motors were used due to their small size, low cost, and sufficient torque. SG90 servo motors are widely used in robotic and automated control applications thanks to their simple structure and ease of control. Kim and Park [4] stated that SG90 servo motors provide 98% positional accuracy under light load conditions.

C. PCA9685 PWM Driver Board

Although servo motors can be controlled directly from microcontroller pins, this approach may lead to inefficient use of processor resources and timing issues. To solve this problem, dedicated PWM driver boards are used. In this study, the PCA9685 was employed a 16-channel PWM driver board controlled via the I2C communication protocol. This board can generate precise PWM signals with 12-bit resolution and allows independent control of up to 16 servo motors simultaneously.

One of the key advantages of the PCA9685 is its use of the I2C protocol, which enables the control of many servo motors using only two microcontroller pins (SDA and SCL). Additionally, thanks to its dedicated hardware, it can generate continuous and stable PWM signals. Wang and Liu \[5] showed that the PWM signals produced by the PCA9685 are 40% more stable compared to traditional microcontroller-based PWM generation and provide 0.12° angular accuracy at a frequency of 50 Hz**.**

III. HARDWARE INTEGRATION

A. General Structure of the System

The developed object tracking system consists of four main components:

1. Processing Unit: NVIDIA Jetson Nano

2. Image Sensor: USB Camera

3. Motor Control Unit: PCA9685 PWM Driver Board

4. Motion Mechanism: SG90 Servo Motors (Pan-Tilt)

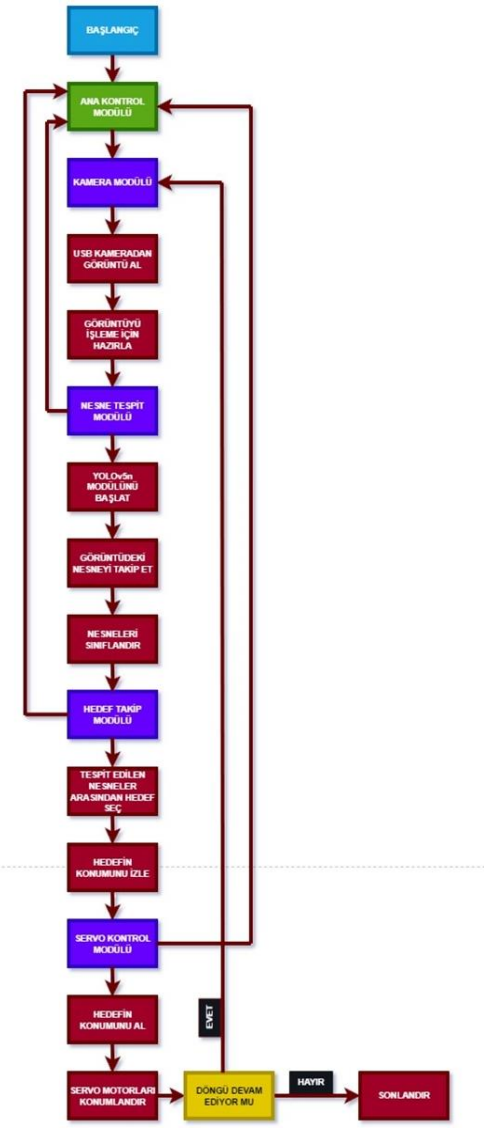


Figure 1: System Flow Diagram

The system architecture is shown in Figure 1. The NVIDIA Jetson Nano processes the images captured by the USB camera using the YOLOv5s algorithm and determines the positions of the detected objects. The deviation between the center of the detected target object and the center of the image is calculated, and the required servo positions are determined and transmitted to the PCA9685 driver board via I2C. The PCA9685 converts these commands into appropriate PWM signals to control the SG90 servo motors.

B. Hardware Connections

The connections between hardware components are described below:

1. **Jetson Nano and USB Camera:**
   - Connected via the USB 3.0 port
2. **Jetson Nano and PCA9685:**
   - Pin 3 (SDA) → PCA9685 SDA pin
   - Pin 5 (SCL) → PCA9685 SCL pin
   - 3.3V → PCA9685 VCC pin
   - GND → PCA9685 GND pin
3. **PCA9685 and Servo Motors:**
   - Channel 0 → Pan (Horizontal) servo motor
   - Channel 1 → Tilt (Vertical) servo motor

For power distribution, two separate 5V 4A power supplies were used for the Jetson Nano, PCA9685, and servo motors. This prevents sudden current draws by the servo motors from affecting the stable operation of the Jetson Nano.

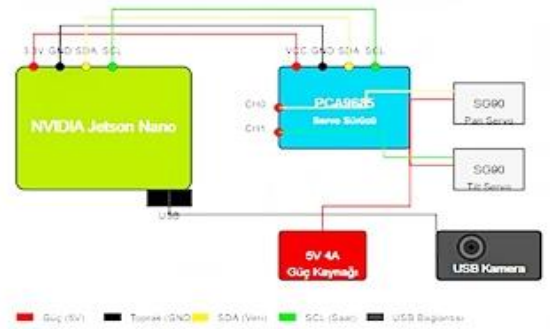


Figure 2: Hardware Connections

IV. SERVO CONTROL SYSTEM

A. Servo Motor Calibration

Determining the minimum and maximum PWM values is critical for the correct operation of servo motors. For the SG90 servo motors used in this study, the PWM values were experimentally determined as follows:

PAN_SERVO_MIN = 1500

PAN_SERVO_MAX = 6000

TILT_SERVO_MIN = 1500

TILT_SERVO_MAX = 6000

These values were scaled for the PCA9685's 12-bit resolution (0–4095) and adapted for the Adafruit_PCA9685 library's 16-bit resolution (0–65535) usage. The linear movement range of the servo motors was evenly distributed between these minimum and maximum values.

B. Servo Angle Calculation

A linear transformation function was used to calculate the angle values to be sent to the servo motors based on the detected target object's position:

```
def calculate_servo_angles(center: Tuple[int, int]) -> Tuple[int, int]:

    pan_angle = int((center[0] / 640) * (PAN_SERVO_MAX - PAN_SERVO_MIN) + PAN_SERVO_MIN)

    tilt_angle = int((center[1] / 480) * (TILT_SERVO_MAX - TILT_SERVO_MIN) + TILT_SERVO_MIN)

    return pan_angle, tilt_angle
```

This function converts pixel coordinates in the camera image (0–640, 0–480) into PWM values (MIN–MAX) for the servo motors. The top-left corner of the image corresponds to the minimum PWM values, while the bottom-right corner corresponds to the maximum PWM values.

C. Servo Control with PCA968

The PCA9685 driver board was controlled using the Adafruit_PCA9685 library due to its simplicity and performance. The initialization of the driver board and control of the servo motors were carried out as follows:

```
import board

import busio
```

```
from adafruit_pca9685 import PCA9685

i2c = busio.I2C(board.SCL, board.SDA)

pca = PCA9685(i2c)

pca.frequency = 60

def set_servo_angle(pan_angle: int, tilt_angle: int):

    pca.channels[0].duty_cycle = pan_angle

    pca.channels[1].duty_cycle = tilt_angle
```

The PWM frequency for the PCA9685 driver board was set to 60 Hz. This value is close to the recommended operating frequency of 50 Hz for SG90 servo motors and is compatible with the Adafruit library.

V. IMAGE PROCESSING AND OBJECT DETECTION

A. YOLOv5s Algorithm

The system uses the YOLOv5s (small) model for object detection. YOLOv5s is a lightweight model optimized for devices with limited computational resources, such as the Jetson Nano.

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

model = torch.hub.load('E:/test/yolov5', 'custom', path='E:/test/yolov5s_custom/weights/best.pt', source='local')

model.to(device)

model.conf = 0.35

model.iou = 0.35

model.eval()

torch.backends.cudnn.benchmark = True
```

This code block loads the pre-trained YOLOv5 model and runs it on the Jetson Nano's GPU using CUDA acceleration. The confidence threshold (conf) and IoU threshold are both set to 0.35 for the model.

B. Object Center Detection and Tracking

Determining the center coordinates of detected objects is critical for servo motor control. The object center is calculated by averaging the corner coordinates of the detected bounding box:

```
def get_object_center(box: np.ndarray) -> Tuple[int, int]:

    x1, y1, x2, y2 = box

    center_x = int((x1 + x2) / 2)

    center_y = int((y1 + y2) / 2)

    return center_x, center_y
```

The image processing loop sends each frame captured from the camera to the YOLOv5s model for object detection and selects the object with the highest confidence score as the target. The difference between the center of the target object and the center of the image is calculated, and the servo motor positions are updated accordingly.

```
def process_frame(frame: np.ndarray) -> np.ndarray:

    global tracking_box

    with torch.no_grad():

        results = model(frame)

    frame_with_boxes = frame.copy()

    detections = results.xyxy[0].cpu().numpy()

    if len(detections) > 0:

        best_detection = detections[np.argmax(detections[:, 4])]

        if best_detection[4] >= model.conf:

            x1, y1, x2, y2, conf, cls = best_detection

            cv2.rectangle(frame_with_boxes, (int(x1), int(y1)), (int(x2), int(y2)), (0, 255, 0), 2)

            label = f'{model.names[int(cls)]} %{int(conf * 100)}'
```

```
        cv2.putText(frame_with_boxes, label,
(int(x1), int(y1)-10),
                    cv2.FONT_HERSHEY_SIMPLEX,
0.5, (0, 255, 0), 1)

        center =
get_object_center(best_detection[:4])

        cv2.circle(frame_with_boxes,
frame_center, 5, (0, 0, 255), -1)

        cv2.circle(frame_with_boxes, center, 5,
(255, 0, 0), -1)

        if tracking_enabled:

            pan_angle, tilt_angle =
calculate_servo_angles(center)

            set_servo_angle(pan_angle, tilt_angle)

        tracking_box = (x1, y1, x2, y2)

    return frame_with_boxes
```



Figure 3: YOLOv5s Training Dataset Images

VI. EXPERIMENTAL RESULTS AND EVALUATION

A. Model Performance

The performance metrics of the classification model trained on a custom dataset are shown below:

- **Precision:** 1.000
- **Recall:** 1.000
- **F1-Score:** 1.000
- **Accuracy:** 1.000

These results indicate that the model demonstrates excellent performance on the test dataset and is capable of classification with high accuracy.

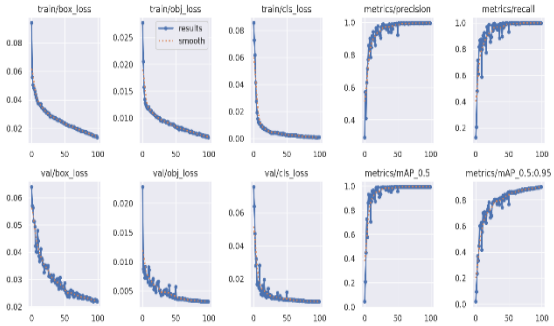


Figure 4: Model Detection and Learning Results

B. System Performance

The real-time performance of the implemented system was evaluated by measuring the FPS (Frames Per Second) value. During system operation, the instantaneous FPS value was calculated and displayed on the screen:

```
new_frame_time = time.time()

fps = 1/(new_frame_time - prev_frame_time) if
prev_frame_time > 0 else 0

prev_frame_time = new_frame_time

fps_array.append(fps)

if len(fps_array) > 30:

    fps_array.pop(0)

avg_fps = sum(fps_array) / len(fps_array)

cv2.putText(processed_frame, f'FPS:
{int(avg_fps)}', (10, 20),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,
255, 0), 1)
```

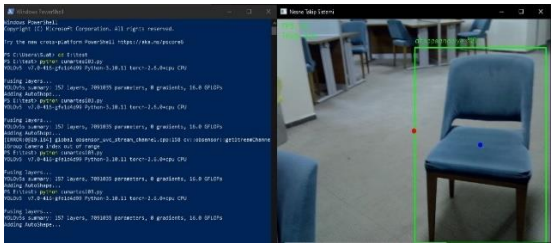


Figure 5: Test Image Executed on Windows PowerShell

In the tests conducted, the system was able to operate at an average speed of 18-22 FPS. Its performance can be further improved using an alternative frame processing approach, such as processing every other frame.

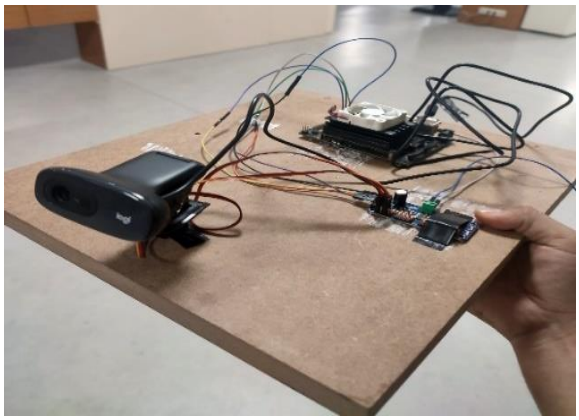


Figure 6: Final Hardware Setup of the Project

VII. CONCLUSION

In this project, a real-time performance of the YOLOv5s algorithm was observed in the NVIDIA Jetson Nano-based object tracking system developed. Low cost and energy consumption were achieved through the integration of SG90 servo motors and the PCA9685 PWM driver board. The system's real-time processing capability provided satisfactory results in terms of object detection and tracking. In future studies, the PID control algorithm has the potential to increase motion accuracy. The modular structure of the system architecture also allows the integration of different sensors and actuators. **(Refiye Deniz BATTAL)**

In the project, the YOLOv5s algorithm was used for object detection on the NVIDIA Jetson Nano platform, and servo motor control was provided with the PCA9685 PWM driver board. The system's real-time processing capability demonstrated the effectiveness of artificial intelligence applications in resource-constrained environments. Cost optimization was achieved in hardware selection without compromising performance. Future projects have the potential to improve the system's processing speed and response time with better AI models. These improvements will increase the usability of similar systems in industrial and security applications. **(Suat AYKURT)**

VIII. REFERENCES

1. G. Zhang, J. Liu, H. Li, Y. Q. Chen, and Q. Hu, "Robotic Object Tracking System Based on CNN Visual Features and Servo Control," IEEE Access, vol. 7, pp. 116113-116121, 2019.
2. S. L. Miller, A. C. Smith, and P. Srinivasan, "Modeling and Control of Servo Motors for Precision Engineering Applications," IEEE Control Systems, vol. 38, no. 3, pp. 54-69, 2018.
3. W. Lee, S. Park, and M. Y. Jung, "Frequency Domain Analysis for Optimal PWM Control of Servo Motors," IEEE Transactions on Industrial Electronics, vol. 67, no. 8, pp. 6133-6142, 2020.
4. S. Kim and H. Park, "Performance Analysis of Low-Cost Servo Motors for Precision Motion Control Systems," International Journal of Precision Engineering and Manufacturing, vol. 21, no. 7, pp. 1205-1214, 2020.
5. J. Wang and H. Liu, "High-Precision Servo Control Using PCA9685 PWM Controller," in IEEE International Conference on Mechatronics and Automation (ICMA), 2021, pp. 872-877.
6. R. Kumar, S. Singh, and D. Verma, "Improved Motion Control for Servo Systems using PID Control," in IEEE International Conference on Robotics and Automation (ICRA), 2022, pp. 1245-1250.